

# Spring Boot Product Service

Complete Project Overview Guide

**Detailed overview of your entire Spring Boot project**

Generated: August 03, 2026 | Java 17 | Spring Boot 4.1

## 1. What Is This Project?

Your **productService** is a **Spring Boot REST API** that acts as a **middle layer (proxy)** between a client (Postman, browser, frontend) and the external **FakeStore API** (fakestoreapi.com). It does **not** use a database. Every product/category request is forwarded to FakeStore, the JSON response is converted into your own Java objects, and sent back to the caller.

### INFO

Think of it like a translator: FakeStore speaks one JSON format, your app converts it into your own **Product** and **Category** models before returning data to the user.

## 2. Technology Stack

| Technology      | Purpose in Your Project                           |
|-----------------|---------------------------------------------------|
| Java 17         | Programming language                              |
| Spring Boot 4.1 | Framework — auto-configures Tomcat, Jackson, DI   |
| Spring Web MVC  | REST controllers, HTTP handling                   |
| RestClient      | Modern HTTP client to call FakeStore API          |
| Lombok          | Auto-generates getters/setters (@Getter, @Setter) |
| Jackson         | Converts JSON ↔ Java objects automatically        |
| Embedded Tomcat | Runs server on port 8080 (default)                |
| Maven (pom.xml) | Dependency management and build tool              |

### 3. Project Structure

Your code is organized into clear layers under **org.pavan.productservice**:

#### **productService/**

- pom.xml → Dependencies & build config
- application.properties → App name (port 8080 default)
- src/main/java/org/pavan/productservice/
  - ProductServiceApplication.java → Main entry point
  - config/RestClientConfig.java → RestClient bean setup
  - controllers/ → REST API endpoints
    - ■■■■ ProductController.java
    - ■■■■ CategoryController.java
    - ■■■■ ExceptionAdvices.java
  - services/ → Business logic layer
    - ■■■■ ProductService.java (interface)
    - ■■■■ FakeStoreProductImplementation.java
    - ■■■■ CategoryService.java (interface)
    - ■■■■ FakeStoreCategoryImplementation.java
  - clients/fakestoreapi/ → HTTP calls to FakeStore
    - ■■■■ FakeStoreProductClient.java
    - ■■■■ FakeStoreCategoryClient.java
  - dtos/ → JSON shape from FakeStore
  - models/ → Your internal domain objects
  - exceptions/ → Custom error handling

## 4. Architecture — How Layers Connect

### Layered Architecture:

#### Client (Postman/Browser)

↓ HTTP Request (GET /products/5)

**Controller Layer** — ProductController, CategoryController

↓ Calls service interface

**Service Layer** — FakeStoreProductImplementation

↓ Converts DTO ↔ Model, calls client

**Client Layer** — FakeStoreProductClient

↓ Uses RestClient

**RestClient** — HTTP call to fakestoreapi.com

↓ JSON response

**Jackson** — JSON → ProductDto → Product → JSON back to client

### Why this structure?

- **Controller** — Only handles HTTP (URLs, status codes, request/response). No business logic.
- **Service** — Business rules and DTO-to-Model conversion. Controller doesn't know about FakeStore.
- **Client** — Only HTTP communication. Service doesn't know URLs or RestClient details.
- **Config** — RestClient base URL defined once, shared by all clients.

## 5. Complete Request Flow (Example)

### GET /products/5 — step by step:

| Step | Component         | What Happens                                             |
|------|-------------------|----------------------------------------------------------|
| 1    | User sends        | GET http://localhost:8080/products/5                     |
| 2    | Tomcat            | Embedded server receives the HTTP request on port 8080   |
| 3    | DispatcherServlet | Spring's front controller routes the request             |
| 4    | HandlerMapping    | Finds ProductController.getSingleProduct(5)              |
| 5    | Controller        | Calls productService.getSingleProduct(5)                 |
| 6    | Service           | Calls fakeStoreProductClient.getSingleProduct(5)         |
| 7    | RestClient        | Sends GET https://fakestoreapi.com/products/5            |
| 8    | FakeStore         | Returns JSON: { id, title, price, image, category, ... } |
| 9    | Jackson           | Converts JSON → ProductDto                               |
| 10   | Service           | Converts ProductDto → Product (your model)               |
| 11   | Controller        | Wraps Product in ResponseEntity, returns 200 OK + JSON   |
| 12   | Client            | Receives JSON product in Postman/browser                 |

#### NOTE

If product ID 999 doesn't exist: FakeStore returns 404. RestClient throws an exception. Your app has no handler for that, so the user sees **500 Internal Server Error** instead of 404.

## 6. Your API Endpoints

### Product Endpoints (/products)

| Method | URL            | What It Does          | Status                                   |
|--------|----------------|-----------------------|------------------------------------------|
| GET    | /products      | Get all products      | Working                                  |
| GET    | /products/{id} | Get one product by ID | Working                                  |
| POST   | /products      | Create new product    | Working (returns 200, should be 201)     |
| PATCH  | /products/{id} | Partial update        | Broken — FakeStore doesn't support PATCH |
| PUT    | /products/{id} | Full replace          | Bug — wrong field mapping in controller  |
| DELETE | /products/{id} | Delete product        | Working                                  |

### Category Endpoints (/categories)

| Method | URL                       | What It Does           | Status                       |
|--------|---------------------------|------------------------|------------------------------|
| GET    | /categories               | Get all category names | Working                      |
| GET    | /categories/{id}/products | Products in category   | Broken — wrong FakeStore URL |
| GET    | /categories/{id}          | Get category by ID     | Stub only (returns string)   |
| POST   | /categories               | Add category           | Stub only                    |
| PUT    | /categories/{id}          | Update category        | Stub only                    |
| DELETE | /categories/{id}          | Delete category        | Stub only                    |

## 7. Key Files — What Each One Does

| File                                 | Purpose                                                                                                    |
|--------------------------------------|------------------------------------------------------------------------------------------------------------|
| ProductServiceApplication.java       | Main class. @SpringBootApplication starts Spring, scans all packages, launches Tomcat on port 8080.        |
| RestClientConfig.java                | Creates a shared RestClient bean with base URL https://fakestoreapi.com. All clients use this single bean. |
| ProductController.java               | Exposes /products REST API. Receives HTTP requests, calls ProductService, returns Product JSON.            |
| CategoryController.java              | Exposes /categories REST API. GET endpoints work; POST/PUT/DELETE are placeholder stubs.                   |
| FakeStoreProductImplementation.java  | Implements ProductService. Calls FakeStoreProductClient and converts ProductDto → Product.                 |
| FakeStoreCategoryImplementation.java | Implements CategoryService. Delegates to FakeStoreCategoryClient.                                          |
| FakeStoreProductClient.java          | Makes actual HTTP calls: GET/POST/PUT/PATCH/DELETE to FakeStore /products endpoints using RestClient.      |
| FakeStoreCategoryClient.java         | HTTP calls for categories. getAllCategories works. getProductsInCategory uses wrong URL.                   |
| ProductService.java                  | Interface defining product operations. Allows swapping FakeStore for a database later.                     |
| CategoryService.java                 | Interface for category operations.                                                                         |
| ProductDto.java                      | Matches FakeStore JSON: id, title, price, description, image, category, rating.                            |
| CategoryDto.java                     | Simple object with category name.                                                                          |
| Product.java                         | Your domain model: title, price, description, Category object, imageUrl.                                   |
| Category.java                        | Domain category with name, description, products list.                                                     |
| BaseModel.java                       | Shared fields: id, createdAt, lastUpdatedAt, isDeleted (for future database use).                          |
| ExceptionAdvices.java                | Global exception handler. Catches NotFoundException → returns 404 JSON.                                    |
| NotFoundException.java               | Custom exception thrown when product is not found.                                                         |
| ErrorResponseDto.java                | Error JSON body: { errorMessage: "..." } returned on 404.                                                  |
| pom.xml                              | Maven config: Spring Boot 4.1, webmvc, lombok, actuator dependencies.                                      |
| application.properties               | Only sets spring.application.name=productService.                                                          |

## 8. RestClient — How Your Project Calls FakeStore

**RestClient** is Spring's modern HTTP client (Spring 6+). You configured it once in `RestClientConfig` with base URL `https://fakestoreapi.com`. Every call in your clients uses a fluent API:

**Pattern used in your project:**

```
restClient.get()           → Start a GET request
.uri("/products/{id}", id) → Set the URL path
.retrieve()               → Send the request
.body(ProductDto.class)   → Convert JSON response to Java object
```

**All calls in FakeStoreProductClient:**

- GET /products → get all products
- GET /products/{id} → get single product
- POST /products → create product
- PUT /products/{id} → replace product
- PATCH /products/{id} → partial update (FakeStore doesn't support this)
- DELETE /products/{id} → delete product

### RestClient vs RestTemplate (quick comparison)

|          | RestClient (You Use)                                                               | RestTemplate (Older)                                                    |
|----------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Status   | Modern, recommended                                                                | Legacy, maintenance mode                                                |
| Style    | Fluent: <code>.get().uri().retrieve().body()</code>                                | Simple: <code>getForObject(url, Class)</code>                           |
| Blocking | Yes (synchronous)                                                                  | Yes (synchronous)                                                       |
| Example  | <code>restClient.get().uri("/products").retrieve().body(ProductDto[].class)</code> | <code>restTemplate.getForObject("/products", ProductDto[].class)</code> |

**INFO**

Use `RestClient` for new projects. `RestTemplate` still works but Spring recommends `RestClient` or `WebClient` for new code.

## 9. DTO vs Model — Why Two Types of Objects?

Your project uses two types of Java objects for the same data. This is a standard industry pattern:

|                | DTO (ProductDto)               | Model (Product)                        |
|----------------|--------------------------------|----------------------------------------|
| Purpose        | Shape of FakeStore JSON        | Your internal business object          |
| Category field | String ("electronics")         | Category object with name, description |
| Image field    | image                          | imageUrl                               |
| Where used     | Client layer (HTTP boundary)   | Controller response, service layer     |
| Who creates    | Jackson auto-deserializes JSON | Service converts from DTO              |

**Conversion flow:** FakeStore JSON → ProductDto (automatic by Jackson) → Product (manual in FakeStoreProductImplementation.convertProductDtoToProduct) → returned to client as JSON.

## 10. Exception Handling Overview

Your project has basic exception handling:

### Current flow:

1. ProductController checks if Optional is empty
2. Throws **NotFoundException** with message
3. **ExceptionHandler** (@ControllerAdvice) catches it
4. Creates **ErrorResponseDto** with error message
5. Returns **404 Not Found** + JSON error body

### What's missing:

- No handler for RestClient errors (FakeStore 404 → your 500)
- No validation errors handler (400 Bad Request)
- No generic 500 handler
- NotFoundException is checked (should be RuntimeException)

## HTTP Status Codes in Your Project:

| Code                      | Meaning          | When It Happens                            |
|---------------------------|------------------|--------------------------------------------|
| 200 OK                    | Success          | GET products, POST product (should be 201) |
| 404 Not Found             | Resource missing | NotFoundException from your code           |
| 500 Internal Server Error | Server crash     | RestClient failure, unhandled exceptions   |

## 11. Spring Boot Concepts Used in Your Project

| Annotation / Concept              | What It Does in Your Project                                                       |
|-----------------------------------|------------------------------------------------------------------------------------|
| @SpringBootApplication            | Starts the app, enables auto-configuration and component scanning.                 |
| @RestController                   | Marks a class as REST API controller. Return values become JSON responses.         |
| @RequestMapping                   | Base URL prefix. '/products' means all methods start with /products.               |
| @GetMapping / @PostMapping / etc. | Maps specific HTTP method + path to a Java method.                                 |
| @PathVariable                     | Reads value from URL. /products/{productId} → productId = 5.                       |
| @RequestBody                      | Converts incoming JSON request body into a Java object.                            |
| @Service                          | Marks business logic class. Spring manages it as a bean.                           |
| @Component                        | Generic Spring-managed bean (used for HTTP clients).                               |
| @Configuration + @Bean            | RestClientConfig creates RestClient as a shared bean.                              |
| @ControllerAdvice                 | Global handler for exceptions across all controllers.                              |
| @ExceptionHandler                 | Method that handles a specific exception type.                                     |
| Dependency Injection              | Constructor receives dependencies. Spring provides them automatically.             |
| IoC Container                     | Spring creates and manages all objects (beans) — you don't use 'new' for services. |

## 12. Issues Found & Recommended Fixes

|                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>ISSUE</b></p> <p><b>ProductController update/replace bug</b></p> <p>Lines 68-70 and 81-83 use <code>product.getTitle()</code> instead of <code>productdto.getTitle()</code>. Updates don't work.</p> |
| <p><b>ISSUE</b></p> <p><b>Duplicate service call</b></p> <p><code>getSingleProduct()</code> called twice in <code>getSingleProduct</code> endpoint (lines 38 and 45).</p>                                  |
| <p><b>ISSUE</b></p> <p><b>Wrong category products URL</b></p> <p><code>FakeStoreCategoryClient</code> uses <code>/categories/{id}/products</code>. Correct: <code>/products/category/{name}</code>.</p>    |
| <p><b>ISSUE</b></p> <p><b>PATCH not supported</b></p> <p><code>FakeStore</code> only supports PUT. Change <code>restClient.patch()</code> to <code>restClient.put()</code>.</p>                            |
| <p><b>ISSUE</b></p> <p><b>404 becomes 500</b></p> <p><code>RestClient</code> throws on <code>FakeStore</code> 404. Add error handling in client layer.</p>                                                 |
| <p><b>ISSUE</b></p> <p><b>POST returns 200</b></p> <p>Should return 201 Created for new resources.</p>                                                                                                     |
| <p><b>ISSUE</b></p> <p><b>Category CRUD stubs</b></p> <p>POST/PUT/DELETE return plain strings, not real operations.</p>                                                                                    |
| <p><b>ISSUE</b></p> <p><b>Main2.java in src/main</b></p> <p>Scratch file — delete it.</p>                                                                                                                  |
| <p><b>ISSUE</b></p> <p><b>Hardcoded FakeStore URL</b></p> <p>Move to <code>application.properties</code>.</p>                                                                                              |
| <p><b>ISSUE</b></p> <p><b>No input validation</b></p> <p>Add <code>@Valid</code> and <code>spring-boot-starter-validation</code>.</p>                                                                      |
| <p><b>ISSUE</b></p> <p><b>No tests</b></p> <p>Only <code>contextLoads</code> test exists. Add controller and client tests.</p>                                                                             |

## 13. How to Run & Test

**Start the application:**

1. Open terminal in project folder
2. Run: `mvn spring-boot:run`
3. Server starts on **`http://localhost:8080`**

**Test with Postman:**

- GET `http://localhost:8080/products` → all products
- GET `http://localhost:8080/products/1` → single product
- POST `http://localhost:8080/products` → create (send JSON body)
- PUT `http://localhost:8080/products/1` → replace product
- DELETE `http://localhost:8080/products/1` → delete product
- GET `http://localhost:8080/categories` → all categories

**Build JAR:** `mvn clean package` → creates `target/productService-0.0.1-SNAPSHOT.jar`

---

This guide covers your complete project at overview level. Read sections 4-5 for architecture understanding, section 6 for API reference, and section 12 for fixes.